Name : Parvinder Kaur
UID: 23BCS11591
Class: 23BCS_KRG-1_B
Subject name: System Design
Subject code: 23CSH-314
Date: 28|02|2026

Experiment 06: To design a Scalable Video Streaming Platform (YouTube-like)

Functional Requirements

- Client should be able to create an account on the platform.
- After successful login, client should be able to upload videos to the platform.
- Client should be able to search for videos based on title, tags, or metadata. – **ELASTIC SEARCH**
- Client should be able to watch videos in multiple different resolutions (360p, 480p, 720p, 1080p etc.) using Adaptive Bitrate Streaming. – **IMPORTANT**
- System should support chunk-based video streaming using HLS and MPEG-DASH protocols.
- Client should be able to like and unlike videos with real-time counter updates.
- Client should be able to post and view comments dynamically with pagination support.
- System should automatically encode uploaded videos into multiple resolutions for dynamic bandwidth handling.

Non-Functional Requirements

- **Scalability:** System should support millions of users and large-scale video storage with horizontal scaling.
- **CAP Theorem: Availability >>> Consistency**
  Availability is critical during video streaming.
  Eventual consistency is acceptable for likes and comments.
- **Latency:** 50–100 ms.
  Client should experience zero or negligible buffering.
- **High Throughput:** System should handle millions of concurrent video views and dynamic interactions.
- **Fault Tolerance:** No single point of failure.
- **Adaptive Bandwidth Handling:** System should dynamically switch video quality (360p, 480p, 720p, 1080p).

Major Entities

1. **Users**
2. **User Metadata** (Profile details)
3. **Videos**
4. **Video Metadata** (Title, Description, Thumbnail, Tags, Resolution, Status)
5. **Video Segments** (Chunks for HLS/DASH streaming)
6. **Likes**
7. **Comments**

API Design

Client-Onboarding

1. POST Call: /api/v1/users/register
2. POST Call: /api/v1/users/login
3. PUT Call: /api/v1/users/update

Video Upload

1. **POST Call:** `/api/v1/videos/upload/init`
   → Returns Pre-Signed URL
2. **POST Call:** /api/v1/videos/upload/complete
3. **GET Call:** `/api/v1/videos/{video_id}`
   → Response: Video Metadata (JSON)

Searching & Video Playing

1. **GET Call:** `/api/v1/search?q={video_title}`
   → Response: List<Video_ID> + Metadata + Pagination
2. **GET Call:** `/api/v1/videos/{video_id}`
   → Response: Video Metadata (JSON)
3. **GET Call:** `/api/v1/videos/{video_id}/manifest.m3u8`
   → HLS Streaming
4. **GET Call:** `/api/v1/videos/{video_id}/manifest.mpd`
   → DASH Streaming

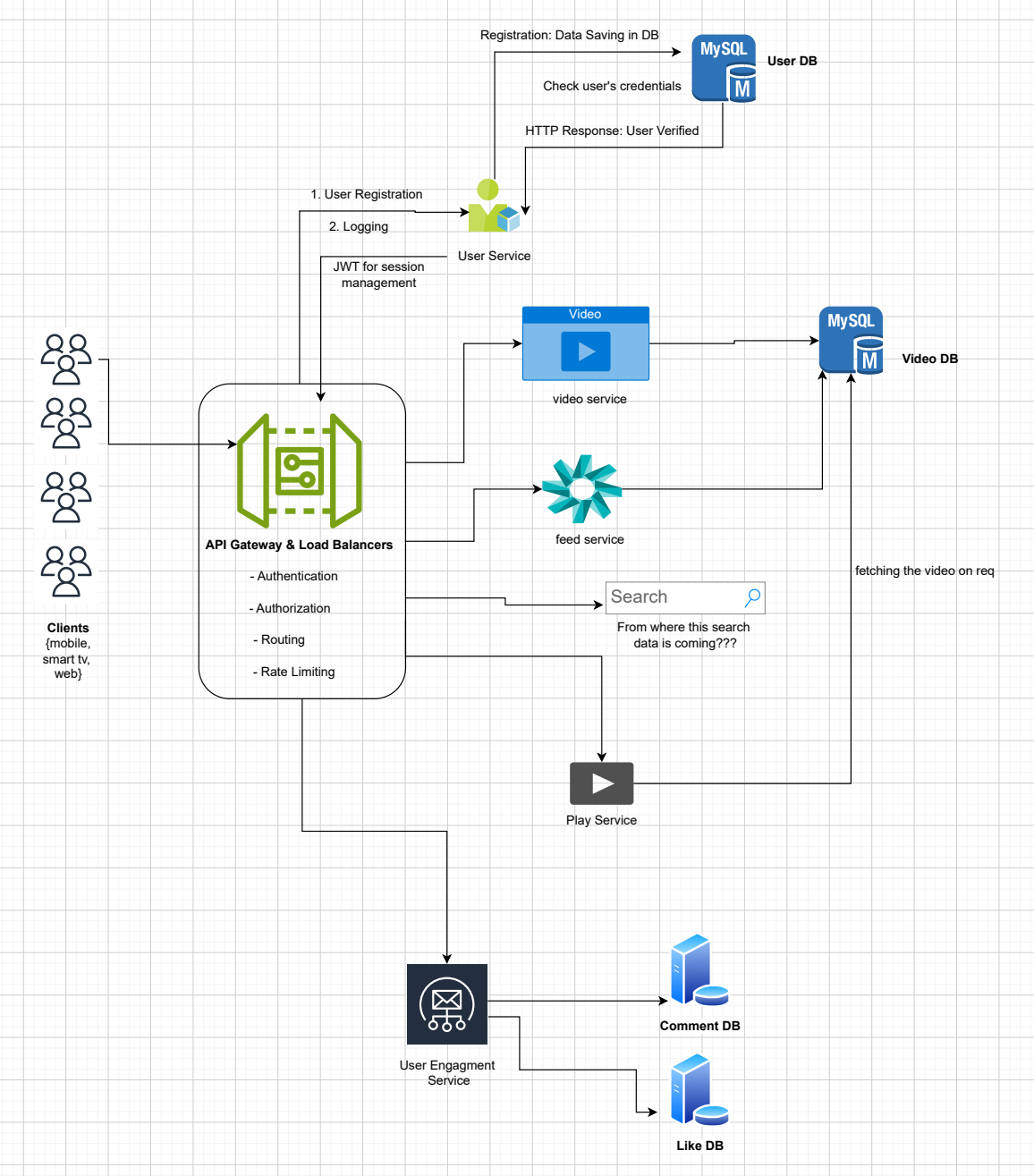
Like & Comment Handling

1. **POST Call:** `/api/v1/videos/{video_id}/like`
2. **DELETE Call:** `/api/v1/videos/{video_id}/like`
3. **POST Call:** `/api/v1/videos/{video_id}/comments`

```
{
  userID,
  commentText
}
```

4. **GET Call:** `/api/v1/videos/{video_id}/comments?page=1`

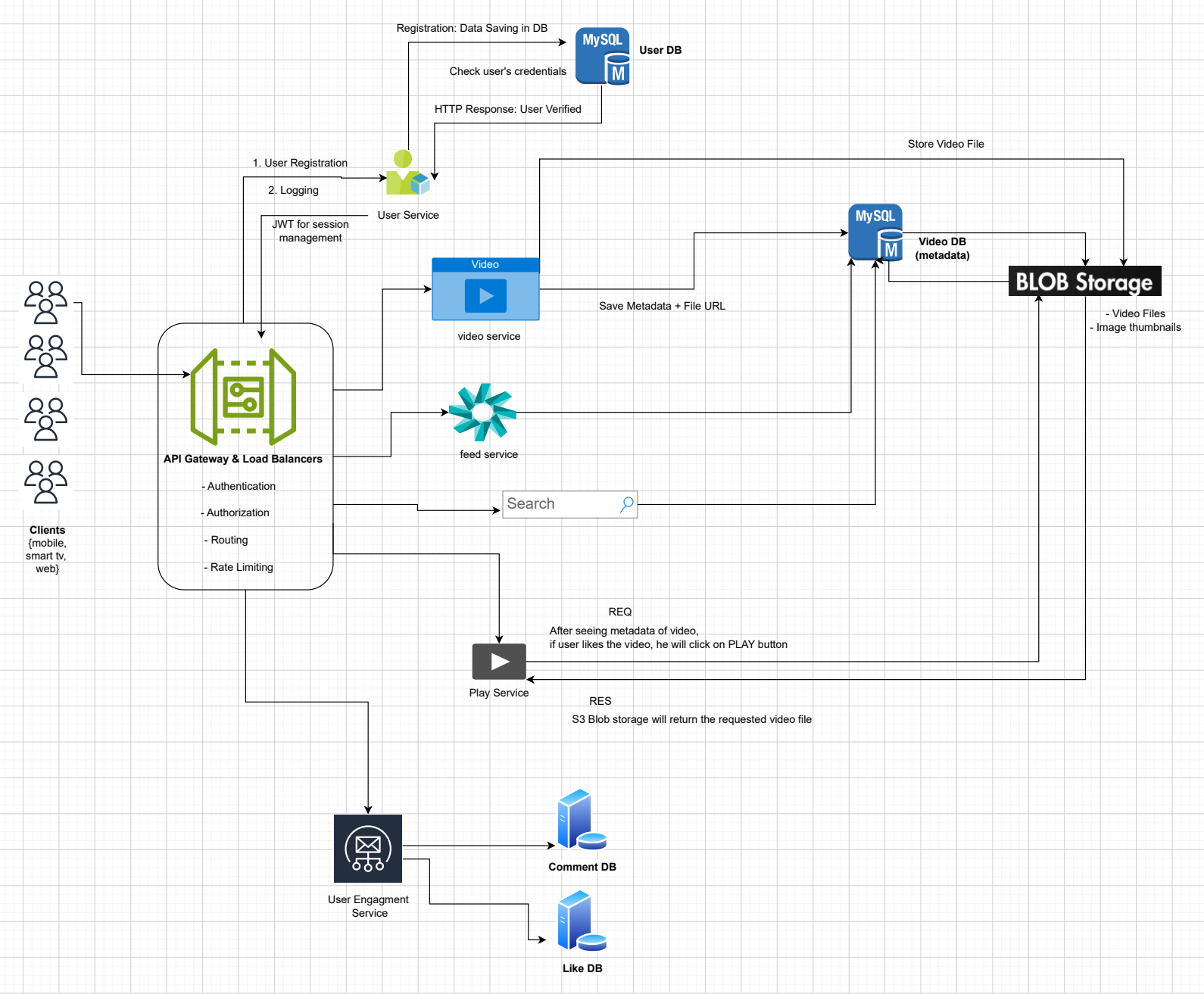
HLD: OTT


Registration: Data Saving in DB
User DB
Check user's credentials
HTTP Response: User Verified
1. User Registration
2. Logging
User Service
JWT for session management
Video
video service
Video DB
feed service
API Gateway & Load Balancers
- Authentication
- Authorization
- Routing
- Rate Limiting
Search
From where this search data is coming???
fetching the video on req
Clients (mobile, smart tv, web)
Play Service
User Engagment Service
Comment DB
Like DB

Problem: Video files are stored in DB >> overload Mysql >> high latency

Solution: Blob Storage (S3-like storage)

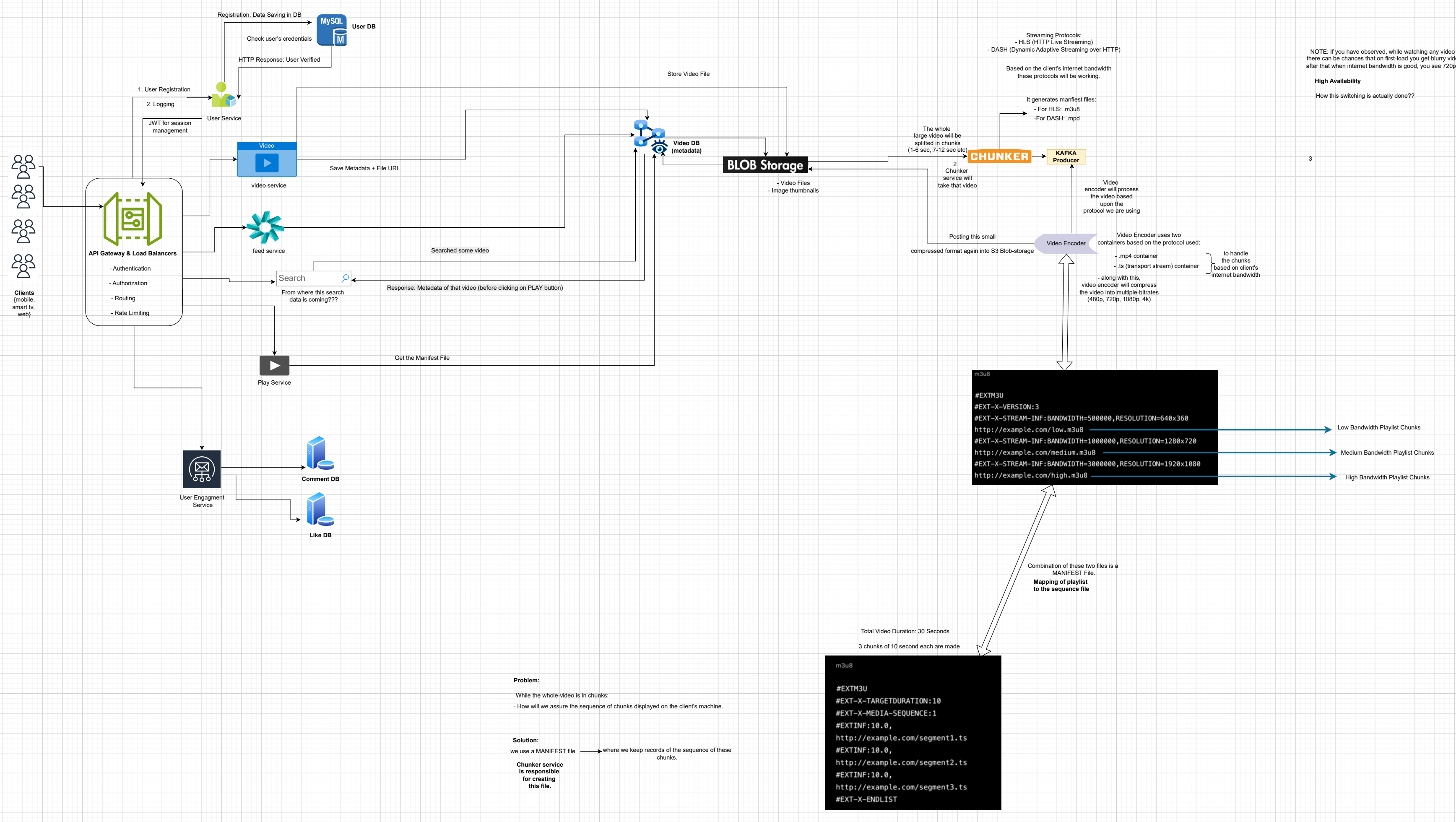
- Video DB → only metadata
- Blob Storage → actual video files


Registration: Data Saving in DB
User DB
Check user's credentials
HTTP Response: User Verified
1. User Registration
2. Logging
User Service
JWT for session management
Store Video File
Video
video service
Save Metadata + File URL
Video DB (metadata)
BLOB Storage
- Video Files
- Image thumbnails
feed service
API Gateway & Load Balancers
- Authentication
- Authorization
- Routing
- Rate Limiting
Search
Clients (mobile, smart tv, web)
REQ
After seeing metadata of video, if user likes the video, he will click on PLAY button
Play Service
RES
S3 Blob storage will return the requested video file
User Engagment Service
Comment DB
Like DB

Problem: right now, we are sending the full video at once and how the resolution changes with respect to internet

Solution: we will introduce media processing layer

this service will:
-convert video into multiple resolutions
-break video into chunks(5-10 sec)
-generate hls/dash


Registration: Data Saving in DB
User DB
Check user's credentials
HTTP Response: User Verified
1. User Registration
2. Logging
User Service
JWT for session management
Store Video File
Video
video service
Save Metadata + File URL
Video DB (metadata)
BLOB Storage
- Video Files
- Image thumbnails
feed service
Searched some video
API Gateway & Load Balancers
- Authentication
- Authorization
- Routing
- Rate Limiting
Search
From where this search data is coming???
Response: Metadata of that video (before clicking on PLAY button)
Clients (mobile, smart tv, web)
Play Service
Get the Manifest File
User Engagment Service
Comment DB
Like DB
The whole large video will be splitted in chunks (1-6 sec, 7-12 sec etc)
2 Chunker service will take that video
CHUNKER
KAFKA Producer
Video Encoder
Posting this small compressed format again into S3 Blob-storage
m3u8
Low Bandwidth Playlist Chunks
Medium Bandwidth Playlist Chunks
High Bandwidth Playlist Chunks
Total Video Duration: 30 Seconds
3 chunks of 10 second each are made
m3u8

Streaming Protocols:
- HLS (HTTP Live Streaming)
- DASH (Dynamic Adaptive Streaming over HTTP)

Based on the client's internet bandwidth these protocols will be working.

It generates manfiest files:
- For HLS: .m3u8
-For DASH: .mpd

Video encoder will process the video based upon the protocol we are using

Video Encoder uses two containers based on the protocol used:
- .mp4 container
- .ts (transport stream) container

to handle the chunks based on client's internet bandwidth

- along with this, video encoder will compress the video into multiple-bitrates (480p, 720p, 1080p, 4k)

```
#EXTM3U
#EXT-X-VERSION:3
#EXT-X-STREAM-INF:BANDWIDTH=500000,RESOLUTION=640x360
http://example.com/low.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=1000000,RESOLUTION=1280x720
http://example.com/medium.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=3000000,RESOLUTION=1920x1080
http://example.com/high.m3u8
```

Combination of these two files is a MANIFEST File.
Mapping of playlist to the sequence file

```
#EXTM3U
#EXT-X-TARGETDURATION:10
#EXT-X-MEDIA-SEQUENCE:1
#EXTINF:10.0,
http://example.com/segment1.ts
#EXTINF:10.0,
http://example.com/segment2.ts
#EXTINF:10.0,
http://example.com/segment3.ts
#EXT-X-ENDLIST
```

Problem:

While the whole-video is in chunks:
- How will we assure the sequence of chunks displayed on the client's machine.

Solution:

we use a MANIFEST file → where we keep records of the sequence of these chunks.

Chunker service is responsible for creating this file.

NOTE: If you have observed, while watching any video on Youtube, there can be chances that on first-load you get blurry video for 5-6 sec, after that when internet bandwidth is good, you see 720p automatically.

before the video is displayed
on our devices, it has to go from 'n' no of operations.

High Availability

How this switching is actually done??

3

Adaptive Streaming Process in YouTube

Imagine YouTube's video processing system creates multiple versions of the same video:

1. **High Quality** (1080p / 4K – for fast Wi-Fi or broadband)
2. **Medium Quality** (720p / 480p – for standard 4G/5G)
3. **Low Quality** (360p / 240p – for slow or unstable networks)

Each version is divided into small chunks (for example, 5–10 second segments)

The Process

1. When you click **Play**, YouTube does not send the entire video at once.
2. Instead, your video player (the client) downloads a small chunk of the video.
3. The player continuously monitors:
   - Your internet speed
   - Buffer size
   - Device performance
4. If your internet speed is fast, it requests the **high-quality chunk**.
5. If your internet speed drops, the player automatically switches and requests a **lower-quality chunk** for the next segment.
6. If your speed improves again, it switches back to a higher resolution.

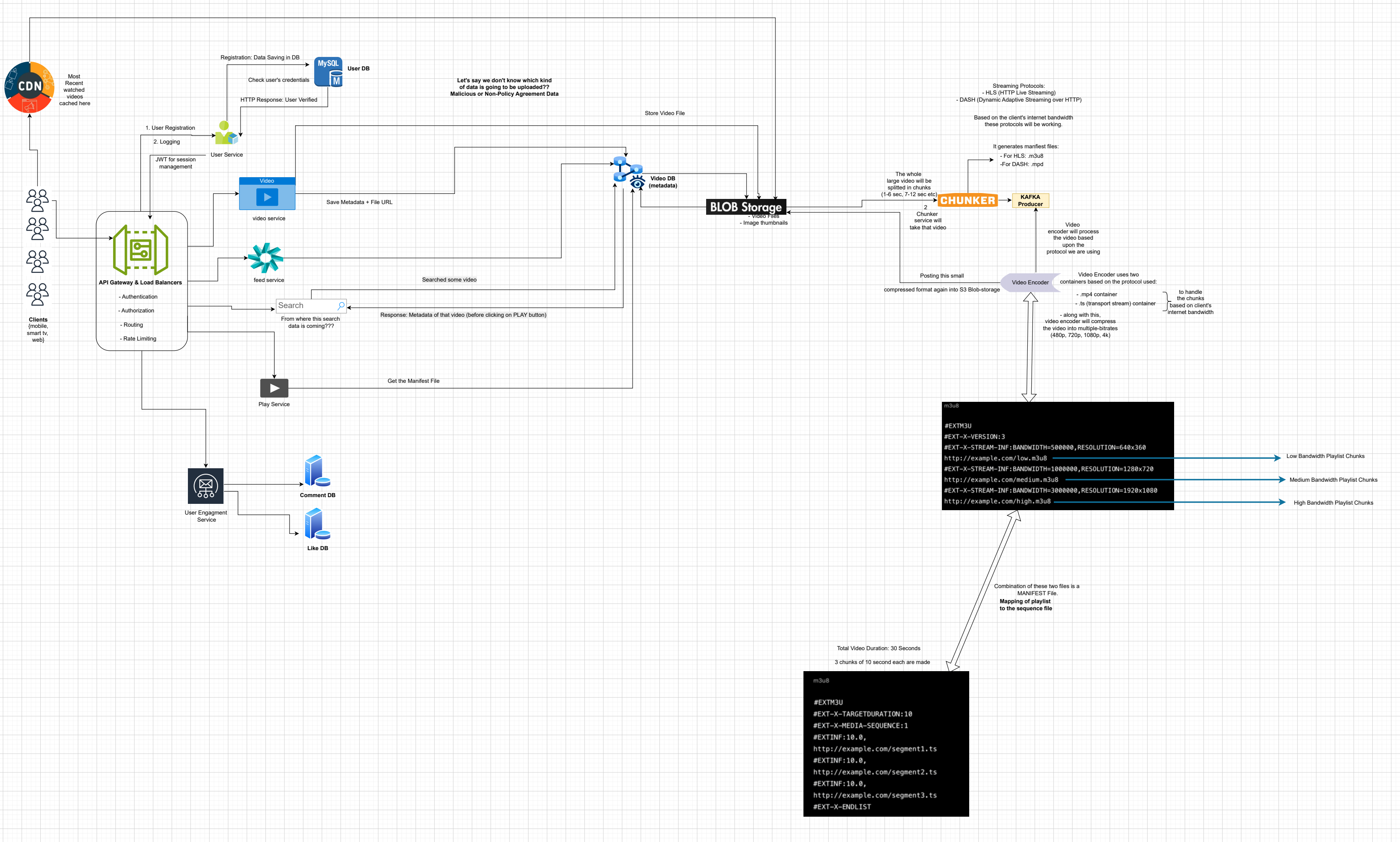
Problem:

Now, Play Service will get manifest file from Video DB and will give back to the client

This manifest file will calculate the bandwidth on client side, and the video / show based upon the bandwidth will be fetched from S3 storage.

but if, you observe clearly, there is a huge gap btw Clients and S3 to fetch the videos.

Solution: To use CDN(Content Delivery Network)


CDN
Most Recent watched videos cached here
Registration: Data Saving in DB
User DB
Check user's credentials
HTTP Response: User Verified
1. User Registration
2. Logging
User Service
JWT for session management
Store Video File
Video
video service
Save Metadata + File URL
Video DB (metadata)
BLOB Storage
- Video Files
- Image thumbnails
feed service
Searched some video
API Gateway & Load Balancers
- Authentication
- Authorization
- Routing
- Rate Limiting
Search
From where this search data is coming???
Response: Metadata of that video (before clicking on PLAY button)
Clients (mobile, smart tv, web)
Play Service
Get the Manifest File
User Engagment Service
Comment DB
Like DB
The whole large video will be splitted in chunks (1-6 sec, 7-12 sec etc)
2 Chunker service will take that video
CHUNKER
KAFKA Producer
Video Encoder
Posting this small compressed format again into S3 Blob-storage
m3u8
Low Bandwidth Playlist Chunks
Medium Bandwidth Playlist Chunks
High Bandwidth Playlist Chunks
Total Video Duration: 30 Seconds
3 chunks of 10 second each are made
m3u8

Let's say we don't know which kind of data is going to be uploaded?? Malicious or Non-Policy Agreement Data

Streaming Protocols:
- HLS (HTTP Live Streaming)
- DASH (Dynamic Adaptive Streaming over HTTP)

Based on the client's internet bandwidth these protocols will be working.

It generates manfiest files:
- For HLS: .m3u8
-For DASH: .mpd

Video encoder will process the video based upon the protocol we are using

Video Encoder uses two containers based on the protocol used:
- .mp4 container
- .ts (transport stream) container

to handle the chunks based on client's internet bandwidth

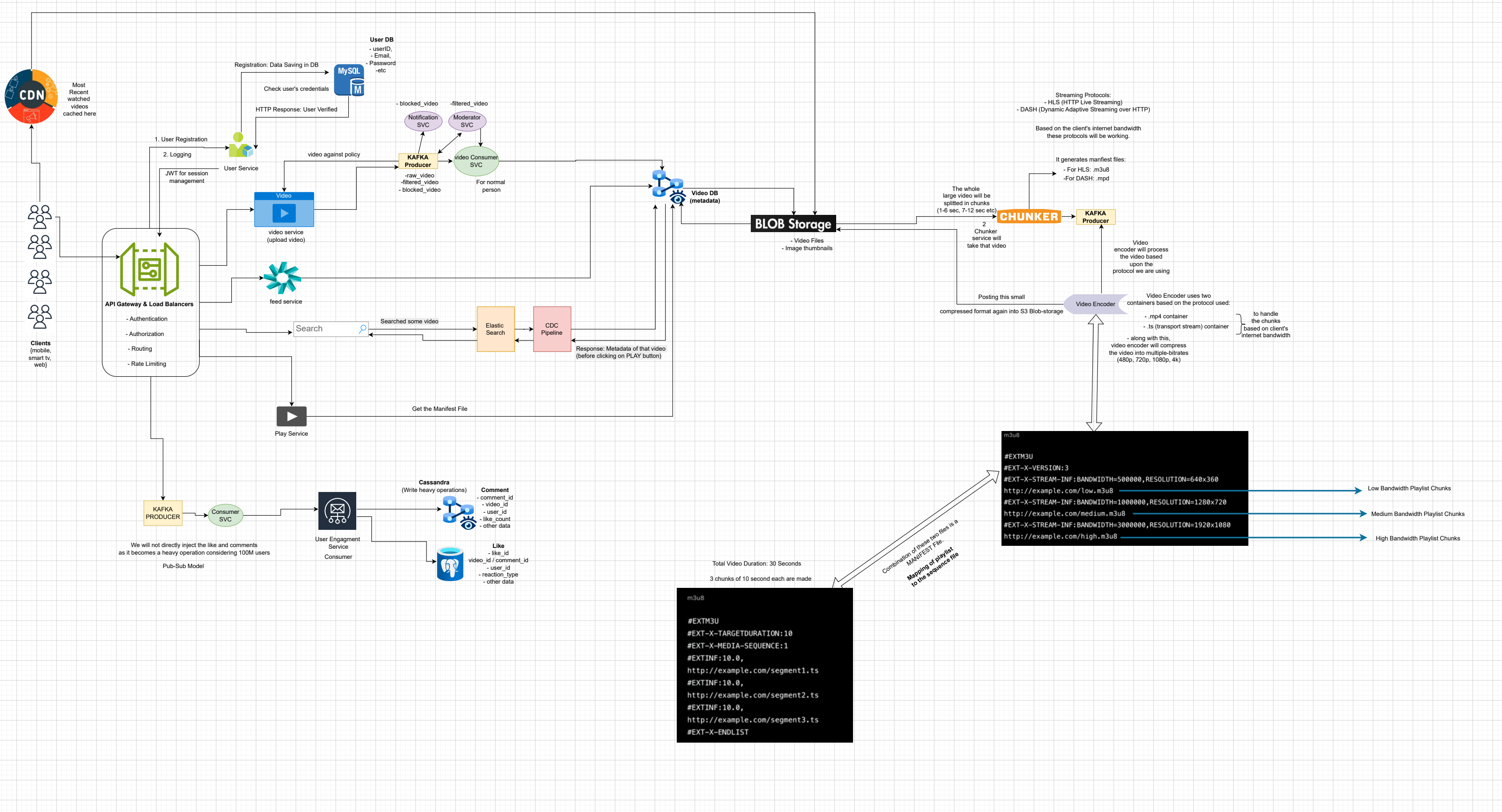
- along with this, video encoder will compress the video into multiple-bitrates (480p, 720p, 1080p, 4k)

```
#EXTM3U
#EXT-X-VERSION:3
#EXT-X-STREAM-INF:BANDWIDTH=500000,RESOLUTION=640x360
http://example.com/low.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=1000000,RESOLUTION=1280x720
http://example.com/medium.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=3000000,RESOLUTION=1920x1080
http://example.com/high.m3u8
```

Combination of these two files is a MANIFEST File.
Mapping of playlist to the sequence file

```
#EXTM3U
#EXT-X-TARGETDURATION:10
#EXT-X-MEDIA-SEQUENCE:1
#EXTINF:10.0,
http://example.com/segment1.ts
#EXTINF:10.0,
http://example.com/segment2.ts
#EXTINF:10.0,
http://example.com/segment3.ts
#EXT-X-ENDLIST
```

Search Service


CDN
Most Recent watched videos cached here
User DB
- userID,
- Email,
- Password
-etc
Registration: Data Saving in DB
Check user's credentials
HTTP Response: User Verified
- blocked_video
-filtered_video
Notification SVC
Moderator SVC
1. User Registration
2. Logging
User Service
video against policy
KAFKA Producer
video Consumer SVC
-raw_video
-filtered_video
- blocked_video
For normal person
JWT for session management
Video
video service (upload video)
Video DB (metadata)
BLOB Storage
- Video Files
- Image thumbnails
feed service
Searched some video
Search
Elastic Search
CDC Pipeline
Response: Metadata of that video (before clicking on PLAY button)
API Gateway & Load Balancers
- Authentication
- Authorization
- Routing
- Rate Limiting
Clients (mobile, smart tv, web)
Play Service
Get the Manifest File
KAFKA PRODUCER
Consumer SVC
User Engagment Service
Consumer
Cassandra (Write heavy operations)
Comment
- comment_id
- video_id
- user_id
- like_count
- other data
Like
- like_id
video_id / comment_id
- user_id
- reaction_type
- other data
The whole large video will be splitted in chunks (1-6 sec, 7-12 sec etc)
2 Chunker service will take that video
CHUNKER
KAFKA Producer
Video Encoder
Posting this small compressed format again into S3 Blob-storage
m3u8
Low Bandwidth Playlist Chunks
Medium Bandwidth Playlist Chunks
High Bandwidth Playlist Chunks
Total Video Duration: 30 Seconds
3 chunks of 10 second each are made
m3u8

We will not directly inject the like and comments as it becomes a heavy operation considering 100M users

Pub-Sub Model

Streaming Protocols:
- HLS (HTTP Live Streaming)
- DASH (Dynamic Adaptive Streaming over HTTP)

Based on the client's internet bandwidth these protocols will be working.

It generates manfiest files:
- For HLS: .m3u8
-For DASH: .mpd

Video encoder will process the video based upon the protocol we are using

Video Encoder uses two containers based on the protocol used:
- .mp4 container
- .ts (transport stream) container

to handle the chunks based on client's internet bandwidth

- along with this, video encoder will compress the video into multiple-bitrates (480p, 720p, 1080p, 4k)

```
#EXTM3U
#EXT-X-VERSION:3
#EXT-X-STREAM-INF:BANDWIDTH=500000,RESOLUTION=640x360
http://example.com/low.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=1000000,RESOLUTION=1280x720
http://example.com/medium.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=3000000,RESOLUTION=1920x1080
http://example.com/high.m3u8
```

Combination of these two files is a MANIFEST File.
Mapping of playlist to the sequence file

```
#EXTM3U
#EXT-X-TARGETDURATION:10
#EXT-X-MEDIA-SEQUENCE:1
#EXTINF:10.0,
http://example.com/segment1.ts
#EXTINF:10.0,
http://example.com/segment2.ts
#EXTINF:10.0,
http://example.com/segment3.ts
#EXT-X-ENDLIST
```

The Feed Service has been included in the diagram only for completeness and architectural representation. Since a detailed High-Level Design for the Feed Service was not part of the given requirements, it has not been elaborated further to avoid unnecessary complexity and length.